JavaScript — initiation au web interactif

Informatique — Licence 1 (L1)

Cours INF-L1-06

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Donner vie aux pages web avec JavaScript : variables et types, conditions, boucles et fonctions, manipulation du DOM, événements et formulaires, tableaux et objets, jusqu'à un projet complet de page interactive validée.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — JavaScript dans la page : scripts et console
• Leçon 2 — Variables, types et opérateurs en JavaScript
• Leçon 3 — Conditions, boucles et fonctions
• Leçon 4 — Le DOM : sélectionner et modifier les éléments
• Leçon 5 — Les événements : clics, saisies et formulaires
• Leçon 6 — Tableaux et objets en JavaScript
• Leçon 7 — Projet : page interactive avec validation de formulaire
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Une page web repose sur trois langages qui se partagent le travail : HTML construit le contenu et la structure, CSS s'occupe de l'apparence, et JavaScript apporte le comportement. C'est lui qui réagit quand tu cliques sur un bouton, qui vérifie un formulaire avant son envoi, qui fait défiler un diaporama ou qui calcule un panier d'achat en direct. Sans JavaScript, une page est une affiche ; avec lui, elle devient un outil avec lequel on dialogue. Tous les sites que tu utilises chaque jour, des réseaux sociaux aux services de mobile money, en contiennent des milliers de lignes. C'est aussi le seul langage de programmation que le navigateur comprenne nativement : aucun autre ne peut prétendre à ce rôle dans une page web, ce qui en fait un passage obligé, quel que soit le métier visé ensuite.

La grande chance du débutant, c'est que JavaScript n'exige aucune installation : il est déjà présent dans chaque navigateur, qui l'exécute dès qu'il rencontre une balise script. Un simple éditeur de texte et un navigateur suffisent donc pour tout ce cours, et la console intégrée au navigateur servira de laboratoire permanent pour tester des idées en quelques secondes. Nous commencerons par les bases du langage — variables, conditions, boucles, fonctions, tableaux et objets — qui ressemblent beaucoup à l'algorithmique déjà étudiée, avant d'attaquer ce qui fait l'originalité du web : le DOM et les événements. Chaque leçon se termine par un exercice corrigé, et le cours se conclut par un projet complet, une page d'inscription avec validation de formulaire, qui assemblera toutes les pièces une à une.

Ce cours suppose que tu connais les balises HTML essentielles (titres, paragraphes, listes, liens, formulaires) et les notions d'algorithmique du cours INF-L1-01 : variable, condition, boucle et sous-programme. Si ces mots te parlent, tu as tout ce qu'il faut. Travaille de façon active : recopie chaque exemple dans un vrai fichier, exécute-le, modifie-le, casse-le volontairement pour voir le message d'erreur. C'est en expérimentant dans la console, ligne après ligne, que JavaScript cesse d'être une liste de règles pour devenir un réflexe. Garde aussi un carnet — papier ou fichier — où tu notes chaque erreur rencontrée et sa cause : en quelques semaines, tu auras constitué ton propre guide de dépannage, et tu reconnaîtras d'un coup d'œil les fautes qui faisaient perdre des heures au début. C'est exactement ainsi que travaillent les développeurs expérimentés.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Insérer du JavaScript dans une page HTML (script interne ou fichier externe) et utiliser la console du navigateur pour tester son code et lire les erreurs.
- Déclarer des variables avec let et const, et manipuler les types de base : nombres, chaînes de caractères et booléens.
- Utiliser les opérateurs arithmétiques, de comparaison (== et ===) et logiques sans tomber dans les pièges de conversion.
- Écrire des conditions et des boucles, et organiser son code en fonctions réutilisables avec paramètres et valeur de retour.
- Sélectionner des éléments du DOM et modifier leur contenu, leurs attributs, leurs classes et leurs styles.
- Réagir aux événements de l'utilisateur (clic, saisie, envoi de formulaire) avec addEventListener et lire les valeurs des champs.
- Valider un formulaire côté client avec des messages d'erreur clairs, et assembler toutes ces notions dans une page interactive complète.

Leçon 1 — JavaScript dans la page : scripts et console

JavaScript s'insère dans une page de deux façons. La première consiste à écrire le code directement dans le HTML, entre les balises <script> et </script> : c'est pratique pour de petits tests. La seconde, recommandée dès que le code grandit, place le code dans un fichier séparé portant l'extension .js, que la page charge avec <script src="app.js"></script>. Le navigateur lit la page de haut en bas et exécute chaque script au moment où il le rencontre. Conséquence importante : un script placé dans l'en-tête qui cherche un bouton situé plus bas dans la page ne le trouvera pas, car ce bouton n'existe pas encore. On place donc le plus souvent les scripts juste avant la balise fermante </body>, ou on ajoute l'attribut defer au chargement du fichier externe.

L'outil indispensable du développeur JavaScript est la console, cachée dans les outils de développement du navigateur, que l'on ouvre avec la touche F12 (ou Ctrl+Maj+J). Elle affiche les messages de ton code grâce à console.log(), très utile pour vérifier la

valeur d'une variable à un moment précis, et elle signale les erreurs en rouge, avec le numéro de ligne concerné. Mieux encore : on peut taper du code directement dans la console et le voir s'exécuter immédiatement, ce qui en fait un terrain d'essai parfait pour une expression ou une petite fonction avant de l'intégrer à la page.

Quelques règles d'écriture à adopter dès le départ. Chaque instruction se termine en général par un point-virgule ; JavaScript peut s'en passer, mais l'écrire évite des surprises. Les commentaires commencent par // pour une ligne, ou se placent entre /* et */ pour plusieurs lignes : ils expliquent le pourquoi du code et sont ignorés à l'exécution. JavaScript distingue strictement majuscules et minuscules : total, Total et TOTAL sont trois variables différentes. Enfin, les fonctions alert() et prompt() existent et affichent des boîtes de dialogue, mais les sites modernes les évitent : nous les utiliserons au plus pour un premier essai, puis nous afficherons les résultats directement dans la page.

L'essentiel
Le code JavaScript vit dans une balise <script> ou dans un fichier .js chargé par la page, et il s'exécute au fil de la lecture du HTML. La console du navigateur (F12) est l'atelier du développeur : console.log() pour observer, messages rouges pour corriger.

Exemple — Premier script : affichage dans la console et dans la page

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Mon premier script</title>
</head>
<body>
<h1 id="titre">Bienvenue</h1>

<script>
// Un message pour la console (touche F12 pour la voir)
console.log("Le script démarre.");

let prenom = "Aminata";
console.log("Bonjour " + prenom + " !");

// Un petit calcul vérifié dans la console
let prix = 1500;
let quantite = 3;
console.log("Total : " + prix * quantite + " FCFA");
</script>
</body>
</html>
```

À retenir

- Script interne (<script> dans le HTML) pour les tests, fichier externe (.js) pour les vrais projets.

• Le navigateur exécute le code dans l'ordre de la page : on place les scripts en fin de body ou avec defer.

• console.log() affiche des valeurs dans la console ; les erreurs y apparaissent en rouge avec leur ligne.

Exercice

1) Écris une page HTML minimale contenant un script qui affiche dans la console trois lignes : ton prénom, ta filière et le résultat du calcul 250 × 12. 2) Un camarade place son script dans le <head> et écrit dedans une ligne qui cherche à modifier le titre <h1 id="titre"> situé dans le <body> ; rien ne se passe et la console affiche une erreur disant que l'élément vaut null. Explique la cause et donne deux façons de corriger.

✔ Corrigé

1) La page contient <script> console.log("Moussa"); console.log("Licence 1 Informatique"); console.log(250 * 12); </script> : la console affiche les deux textes puis 3000, car l'expression 250 * 12 est calculée avant l'affichage. 2) Cause : le navigateur exécute le script du <head> avant d'avoir lu le <body> ; le titre n'existe donc pas encore dans la page et sa recherche renvoie null, d'où l'erreur quand on essaie de le modifier. Corrections possibles : déplacer la balise <script> juste avant </body>, de sorte que tout le HTML soit déjà lu ; ou garder le fichier externe dans le <head> en ajoutant l'attribut defer (<script src="app.js" defer></script>), qui repousse l'exécution du script après l'analyse complète de la page. Les deux solutions reposent sur la même idée : le code ne doit chercher un élément que lorsque celui-ci existe.

Leçon 2 — Variables, types et opérateurs en JavaScript

JavaScript connaît trois façons de déclarer une variable, mais deux seulement sont recommandées aujourd'hui : let pour une valeur qui peut changer (un compteur, un total) et const pour une valeur fixée une fois pour toutes dans le programme (un taux, un nom d'école). L'ancien mot-clé var existe encore dans de vieux codes, mais ses règles de portée prêtent à confusion ; nous l'éviterons. Le nom d'une variable se choisit librement, sans espaces ni accents, et la convention est le camelCase : prixTotal, nombreEtudiants. Contrairement au langage C, on ne déclare pas le type : JavaScript le déduit de la valeur rangée dans la variable, et ce type peut même changer en cours de route — souplesse agréable, mais source d'erreurs si l'on n'y prend pas garde.

Les types de base que nous utiliserons sont : le nombre (number), qui couvre entiers et décimaux, comme 42 ou 12,5 ; la chaîne de caractères (string), entre guillemets simples, doubles ou accents graves, comme "Bamako" ; et le booléen (boolean), qui ne vaut que true ou false. Deux valeurs spéciales complètent le tableau : undefined, la valeur d'une variable déclarée mais jamais remplie, et null, l'absence de valeur décidée par le programmeur. La fonction typeof permet d'interroger le type d'une valeur : typeof 42 renvoie "number", typeof "IAM" renvoie "string". Les chaînes construites avec les accents graves, appelées littéraux de gabarit, acceptent d'insérer

des variables grâce à ${ } : `Total : ${total} FCFA` est bien plus lisible qu'une suite de concaténations avec +.

Les opérateurs arithmétiques sont +, -, *, /, % (le reste de la division entière) et ** (la puissance). Attention au piège le plus célèbre du langage : + additionne deux nombres, mais il colle deux chaînes — et si l'un des deux côtés est une chaîne, c'est la colle qui gagne : "5" + 3 donne "53", alors que "5" - 3 donne 2, car - force la conversion en nombre. Un champ de formulaire livre toujours du texte : avant de calculer avec une saisie, on convertit avec Number(). Pour comparer, on dispose de <, >, <=, >=, et surtout de deux égalités : == compare après conversion des types ("5" == 5 est vrai, ce qui surprend), tandis que === exige même valeur et même type ("5" === 5 est faux). En pratique, on utilise presque toujours ===. Enfin, && (et), || (ou) et ! (non) combinent les booléens.

L'essentiel
let pour ce qui change, const pour ce qui est fixé ; le type vient de la valeur. Le + colle dès qu'une chaîne est présente, et les saisies de formulaire arrivent en texte : convertir avec Number(). Pour comparer, préférer === qui vérifie valeur et type.

Exemple — Total d'une petite boutique, avec conversion de la saisie

```
const TAUX_REMISE = 0.10; // 10 % de remise
let nomArticle = "Cahier 200 pages";
let prixUnitaire = 750; // en FCFA
let quantiteTexte = "4"; // comme lu dans un champ : du texte !

let quantite = Number(quantiteTexte); // conversion en nombre
let sousTotal = prixUnitaire * quantite; // 3000
let remise = sousTotal * TAUX_REMISE; // 300
let total = sousTotal - remise; // 2700

console.log(`${nomArticle} x${quantite} : ${total} FCFA`);
console.log(typeof total); // "number"
console.log("5" + 3); // "53" : concaténation
console.log("5" === 5); // false : types différents
```

À retenir

- let = modifiable, const = non réassignable ; éviter var dans du code nouveau.
- Une saisie utilisateur est toujours une chaîne : Number() avant tout calcul.
- === compare valeur ET type ; == convertit les types et réserve des surprises.

Exercice

Sans exécuter le code, prédis ce qu'affiche chaque ligne, puis justifie : a) console.log(10 + "5"); b) console.log("10" - "4"); c) console.log(typeof true); d) let x; console.log(x); e) console.log(7 % 3); f) const VILLE = "Bamako"; VILLE = "Sikasso"; que se passe-t-il ?

✔ Corrigé
a) "105" : le + rencontre une chaîne, il bascule en concaténation et colle le nombre converti en texte. b) 6 : l'opérateur - n'existe pas pour les chaînes, JavaScript convertit donc "10" et "4" en nombres et calcule 10 - 4. c) "boolean" : true est un booléen. d) undefined : x est déclarée mais n'a reçu aucune valeur ; ce n'est pas une erreur, c'est une valeur spéciale qui signale « pas encore remplie ». e) 1 : le reste de la division de 7 par 3 vaut 1 (7 = 3 × 2 + 1). f) Erreur : une constante déclarée avec const ne peut pas recevoir une nouvelle valeur ; le navigateur signale « Assignment to constant variable ». Si la ville doit changer dans le programme, il faut déclarer la variable avec let dès le départ.

Leçon 3 — Conditions, boucles et fonctions

Les conditions de JavaScript ressemblent trait pour trait à celles de l'algorithmique : if (condition) { ... } else if (autre condition) { ... } else { ... }, avec les accolades pour délimiter les blocs. La cascade s'évalue de haut en bas et s'arrête à la première condition vraie, comme dans l'exemple classique des mentions selon la moyenne. Quand on compare une même variable à une série de valeurs précises, le switch offre une écriture plus claire, à condition de ne pas oublier le break qui termine chaque cas. Il existe aussi une forme raccourcie, l'opérateur ternaire condition ? valeurSiVrai : valeurSiFaux, pratique pour choisir entre deux valeurs en une ligne — par exemple afficher « Admis » ou « Ajourné ».

Les boucles reprennent elles aussi les trois formes connues. La boucle for regroupe initialisation, condition et progression dans son en-tête : for (let i = 0; i < 10; i++) est l'outil standard pour répéter un nombre de fois connu, et i++ signifie « ajoute 1 à i ». La boucle while répète tant que sa condition reste vraie, sans nombre de tours fixé à l'avance ; do...while garantit au moins un passage. Le danger reste la boucle infinie, quand rien ne fait évoluer la condition : le navigateur se fige sur l'onglet. Les instructions break (sortir immédiatement) et continue (sauter au tour suivant) donnent un contrôle fin à l'intérieur du corps de boucle.

Les fonctions regroupent un traitement sous un nom pour le réutiliser : function saluer(prenom) { return "Bonjour " + prenom; }. Les paramètres sont les entrées, return fournit la sortie et met fin à l'exécution de la fonction. Une variable déclarée dans une fonction avec let y reste enfermée : c'est sa portée, et c'est une protection contre les conflits de noms. On rencontre aussi une écriture compacte, les fonctions fléchées : const carre = (x) => x * x;. Quelle que soit la forme, la règle d'or ne change pas par rapport aux sous-programmes de l'algorithmique : une fonction fait une seule chose, clairement nommée, et c'est ce découpage qui rendra la validation de formulaires de la fin du cours presque facile.

L'essentiel
if / else if / else pour décider, switch pour les valeurs précises, ternaire pour un choix en une ligne. for quand on compte, while quand on attend une condition. Les fonctions prennent des paramètres, rendent un résultat avec return, et gardent leurs variables chez elles.

Exemple — Mention d'un étudiant et table de multiplication

```
function mention(moyenne) {
if (moyenne >= 16) {
return "Très Bien";
} else if (moyenne >= 14) {
return "Bien";
} else if (moyenne >= 12) {
return "Assez Bien";
} else if (moyenne >= 10) {
return "Passable";
} else {
return "Ajourné";
}
}

console.log(mention(13.5)); // "Assez Bien"
console.log(mention(8)); // "Ajourné"

// Table de multiplication de 7, de 1 à 5
for (let i = 1; i <= 5; i++) {
console.log("7 x " + i + " = " + 7 * i);
}
```

À retenir

- La première condition vraie d'une cascade l'emporte : tester les cas dans le bon ordre.
- Toute boucle doit faire évoluer sa condition, sinon elle tourne pour toujours.
- return rend le résultat ET arrête la fonction : le code placé après ne s'exécute pas.

Exercice

1) Écris une fonction estAdmis(moyenne) qui renvoie true si la moyenne est supérieure ou égale à 10, false sinon, en une seule ligne dans le corps de la fonction. 2) Écris une boucle qui affiche dans la console les nombres pairs de 0 à 20 inclus. 3) Que vaut compteur à la fin de ce code : let compteur = 0; for (let i = 1; i <= 10; i++) { if (i % 2 === 0) { continue; } compteur++; } ? Justifie.

✔ Corrigé

1) function estAdmis(moyenne) { return moyenne >= 10; } — la comparaison produit déjà un booléen, il suffit de le retourner, sans if. 2) for (let i = 0; i <= 20; i += 2) { console.log(i); } : on avance de 2 en 2 depuis 0, on obtient 0, 2, 4, …, 20. Une variante avec i++ et un test i % 2 === 0 donne le même résultat. 3) compteur vaut 5. La boucle fait 10 tours (i de 1 à 10) ; quand i est pair, continue saute l'incrémentation et passe au tour suivant. Seuls les i impairs (1, 3, 5, 7, 9) augmentent compteur, soit 5 fois. L'exercice montre la différence entre break, qui aurait quitté la boucle dès i = 2, et continue, qui ne saute que le reste du tour courant.

Leçon 4 — Le DOM : sélectionner et modifier les éléments

Pour agir sur la page, JavaScript a besoin d'un pont entre le code et le HTML : c'est le DOM (Document Object Model). Quand le navigateur charge une page, il la transforme en un arbre d'objets : un objet document à la racine, puis un objet pour chaque balise — le titre, les paragraphes, les boutons, les champs. Modifier l'arbre revient à modifier ce que l'utilisateur voit, sans recharger la page. Toute l'interactivité du web repose sur cette idée simple : la page affichée n'est qu'une photographie de l'arbre, et JavaScript peut retoucher l'arbre à tout moment.

Première étape : sélectionner l'élément visé. La méthode historique document.getElementById("titre") récupère l'élément dont l'attribut id vaut "titre" — un id étant unique dans la page, la réponse est un seul élément, ou null s'il n'existe pas. La méthode moderne document.querySelector("...") accepte n'importe quel sélecteur CSS : "#titre" pour un id, ".important" pour une classe, "p" pour une balise ; elle renvoie le premier élément correspondant. Sa sœur querySelectorAll renvoie la liste de tous les éléments correspondants, que l'on peut ensuite parcourir avec une boucle. Choisir un bon sélecteur suppose donc de connaître ses sélecteurs CSS.

Deuxième étape : modifier l'élément sélectionné. La propriété textContent remplace son texte, en toute sécurité car le contenu est traité comme du texte brut. La propriété innerHTML fait la même chose en interprétant les balises, ce qui permet d'insérer du HTML, mais devient dangereuse si le texte vient de l'utilisateur (risque d'injection de code) : on la réserve au contenu que l'on maîtrise. On peut aussi changer un attribut avec setAttribute (par exemple la source d'une image), jouer avec les styles via element.style (element.style.color = "red"), ou ajouter et retirer des classes avec classList.add, classList.remove et classList.toggle — la méthode la plus propre, car la présentation reste dans le CSS. Enfin, document.createElement("li") fabrique un nouvel élément que appendChild accroche dans l'arbre : c'est ainsi que naissent les listes dynamiques.

L'essentiel
Le DOM est la copie vivante de la page sous forme d'arbre d'objets. On sélectionne avec getElementById ou querySelector, puis on modifie : textContent pour le texte, classList pour les classes, style pour les styles, createElement + appendChild pour créer.

Exemple — Personnaliser un titre et allonger une liste de cours

```
<h1 id="titre">Cours du jour</h1>
<ul id="cours">
<li>Algorithmique</li>
<li>Langage C</li>
</ul>

<script>
// Sélectionner et modifier le titre
let titre = document.getElementById("titre");
titre.textContent = "Cours du jour : JavaScript";
titre.style.color = "darkblue";
titre.classList.add("important");

// Créer un nouvel élément de liste et l'ajouter
let nouvelItem = document.createElement("li");
nouvelItem.textContent = "JavaScript";
let liste = document.querySelector("#cours");
liste.appendChild(nouvelItem);

console.log("Nombre de cours : " + liste.children.length); // 3
</script>
```

À retenir

• getElementById vise un id unique ; querySelector utilise la syntaxe des sélecteurs CSS.

• textContent = texte brut et sûr ; innerHTML = HTML interprété, à réserver au contenu maîtrisé.

• classList (add, remove, toggle) est la façon propre de changer l'apparence : le style reste dans le CSS.

Exercice

On donne la page suivante : Bienvenue. <button id="btn">Dire bonjour</button>. Sans utiliser d'événement pour l'instant (ce sera la leçon suivante), écris le script qui, dès le chargement : 1) remplace le texte du paragraphe par « Bonjour les étudiants de L1 ! » ; 2) donne au bouton la classe "actif" ; 3) change la couleur de fond du paragraphe en "lightyellow". Puis explique ce qui se passe si l'id écrit dans getElementById contient une faute de frappe, par exemple "mesage".

✔ Corrigé
Le script : let p = document.getElementById("message"); p.textContent = "Bonjour les étudiants de L1 !"; document.getElementById("btn").classList.add("actif"); p.style.backgroundColor = "lightyellow";. Note le nom de propriété backgroundColor : en JavaScript, les propriétés de style CSS s'écrivent en camelCase, le tiret de background-color disparaît. Si l'id est mal orthographié, getElementById ne trouve rien et renvoie null ; dès la ligne suivante, p.textContent déclenche une erreur du type « Cannot set properties of null », et tout le reste du script s'arrête. Réflexe à prendre : quand un script « ne fait rien », vérifier d'abord dans la console si une sélection n'a pas renvoyé null — id fautif et script exécuté avant l'existence de l'élément sont les deux causes presque exclusives de ce message.

Leçon 5 — Les événements : clics, saisies et formulaires

Jusqu'ici, nos scripts s'exécutaient une fois, de haut en bas, au chargement. Une page interactive fonctionne autrement : elle attend que l'utilisateur agisse, puis réagit. Chaque action — un clic, une touche pressée, un champ modifié, un formulaire envoyé — déclenche un événement. Le travail du programmeur consiste à attacher une fonction à un événement sur un élément : cette fonction, appelée gestionnaire, restera en attente et sera appelée par le navigateur au bon moment. La méthode moderne pour attacher un gestionnaire est addEventListener : bouton.addEventListener("click", traiterClic). On évite d'écrire le code dans le HTML (onclick="...") car mélanger structure et comportement rend les pages difficiles à entretenir.

Les événements les plus utiles au quotidien sont : click, le clic de souris ; input, déclenché à chaque caractère tapé dans un champ, parfait pour réagir en direct ; change, quand la valeur d'un champ est confirmée ; submit, à l'envoi d'un formulaire ; mouseover, au survol ; et keydown, à l'appui d'une touche. Le gestionnaire reçoit automatiquement un objet, souvent nommé event, qui décrit ce qui s'est passé : event.target désigne l'élément qui a déclenché l'événement, ce qui permet d'écrire un seul gestionnaire pour plusieurs éléments. Pour un champ de saisie, la valeur courante se lit dans sa propriété value — et rappelle-toi la leçon 2 : cette valeur est toujours une chaîne de caractères.

Le cas des formulaires mérite une attention particulière. Par défaut, envoyer un formulaire recharge la page et transmet les données au serveur : c'est le comportement historique du HTML. Quand JavaScript veut vérifier les données avant le départ, le gestionnaire de submit appelle event.preventDefault() pour empêcher ce rechargement, puis effectue ses contrôles tranquillement. C'est le mécanisme central de toute validation côté client, que nous exploiterons dans le projet final. Retiens le schéma complet : sélectionner les éléments, attacher les gestionnaires avec addEventListener, lire les valeurs avec .value, vérifier, puis réagir dans la page — messages, classes, modifications du DOM.

L'essentiel
Une page interactive attend des événements (click, input, change, submit…) et y répond par des fonctions attachées avec addEventListener. L'objet event décrit l'action, .value lit un champ, et preventDefault() empêche le rechargement lors d'un envoi de formulaire.

Exemple — Compteur de clics et écho de saisie en direct

```
<button id="btnCompteur">Clique-moi</button>
<p>Nombre de clics : <span id="affichage">0</span></p>

<label>Ton prénom : <input type="text" id="champPrenom"></label>
<p id="echo"></p>

<script>
let compteur = 0;
let bouton = document.getElementById("btnCompteur");
let affichage = document.getElementById("affichage");

bouton.addEventListener("click", function () {
compteur++;
affichage.textContent = compteur;
});

let champ = document.getElementById("champPrenom");
let echo = document.getElementById("echo");
champ.addEventListener("input", function () {
echo.textContent = "Tu écris : " + champ.value;
});
</script>
```

À retenir

- addEventListener("événement", fonction) attache un gestionnaire sans polluer le HTML.
- input réagit à chaque frappe, click à chaque clic, submit à l'envoi du formulaire.
- preventDefault() stoppe le comportement par défaut (le rechargement de page du formulaire).

Exercice

1) Écris le code JavaScript d'un champ mot de passe <input type="password" id="mdp"> accompagné d'un paragraphe : à chaque saisie (événement input), le paragraphe affiche « Mot de passe trop court » si la valeur compte moins de 6 caractères, « Longueur acceptable » sinon. 2) Pourquoi choisir l'événement input plutôt que click pour cet exercice ? 3) Dans un gestionnaire de submit, qu'observe-t-on si l'on oublie preventDefault() ?

✔ Corrigé
1) let mdp = document.getElementById("mdp"); let avis = document.getElementById("avis"); mdp.addEventListener("input", function () { if (mdp.value.length < 6) { avis.textContent = "Mot de passe trop court"; } else { avis.textContent = "Longueur acceptable"; } });. La propriété length d'une chaîne donne son nombre de caractères, ce qui rend le test direct. 2) Le clic ne dit rien du contenu du champ : l'utilisateur peut cliquer une fois puis taper vingt caractères sans nouveau clic. L'événement input, déclenché à chaque modification du texte, garantit que l'avis suit la saisie en temps réel — c'est exactement le retour immédiat attendu pendant qu'on tape un mot de passe. 3) Sans preventDefault(), le navigateur exécute son comportement par défaut dès l'envoi : la page se recharge, les messages affichés par le script disparaissent aussitôt et les valeurs repartent de zéro. La validation semble alors « ne pas marcher », alors qu'elle a bien eu lieu, une fraction de seconde avant le rechargement.

Leçon 6 — Tableaux et objets en JavaScript

Un tableau regroupe plusieurs valeurs sous un seul nom, dans des cases numérotées à partir de 0 : let notes = [12, 9, 15, 10];. On lit une case par son indice (notes[0] vaut 12), on connaît le nombre d'éléments par notes.length, et on ajoute une valeur en fin de tableau avec push : notes.push(14). Les méthodes pop (retirer le dernier élément), indexOf (trouver la position d'une valeur) et includes (tester une présence) complètent la boîte à outils de base. Le piège symétrique à celui de beaucoup de langages : le dernier indice vaut length - 1, et demander notes[4] quand il n'y a que quatre éléments renvoie undefined, sans erreur fracassante.

Parcourir un tableau est le geste le plus fréquent de la programmation web : afficher une liste de produits, calculer une moyenne, chercher un élément. Trois écritures dominent : la boucle for classique avec indice, utile quand la position compte ; la boucle for...of, qui donne directement chaque valeur sans manipuler d'indices — for (let note of notes) { ... } — parfaite pour une simple lecture ; et la méthode forEach, qui applique une fonction à chaque élément. Pour transformer ou filtrer un tableau, les méthodes map et filter créent un nouveau tableau sans toucher à l'original : les notes au-dessus de 10 s'obtiennent par notes.filter(n => n >= 10). À ce niveau, savoir lire ces écritures suffit ; les produire viendra avec la pratique.

Quand les données décrivent une entité — un étudiant, un produit, un livre — les ranger dans des cases numérotées devient maladroit : on utilise un objet, c'est-à-dire un ensemble de paires clé/valeur entre accolades : let etudiant = { nom: "Fatou", age: 19, moyenne: 13.5 };. On lit une propriété par la notation pointée etudiant.nom, ou par crochets etudiant["nom"], équivalente et utile quand la clé est dans une variable. Les objets se combinent avec les tableaux : un tableau d'objets représente une liste d'étudiants, chaque ligne étant une fiche complète. Ce format de données est omniprésent, car c'est aussi la forme du JSON, le langage d'échange entre pages web et serveurs : maîtriser les objets, c'est déjà préparer le travail avec les données réelles.

L'essentiel
Un tableau range des valeurs sous des indices qui commencent à 0 ; length donne sa taille, push ajoute un élément. On le parcourt avec for, for...of ou forEach. Un objet regroupe des propriétés nommées (clé: valeur) pour décrire une entité, et les tableaux d'objets décrivent des collections.

Exemple — Moyenne d'un tableau de notes et fiche d'un étudiant

```
let notes = [12, 9, 15, 10, 14];
let somme = 0;
for (let note of notes) {
somme += note;
}
let moyenne = somme / notes.length;
console.log("Moyenne : " + moyenne); // 12

let admis = notes.filter(n => n >= 10);
console.log("Notes admises : " + admis.length); // 4

let etudiant = { nom: "Fatou", age: 19, moyenne: moyenne };
console.log(etudiant.nom + " a " + etudiant.age + " ans");
console.log(etudiant["moyenne"]); // même accès, par crochets

etudiant.moyenne = 14.25; // modifier une propriété
etudiant.filiere = "Informatique"; // en ajouter une nouvelle
console.log(etudiant);
```

À retenir

- Les indices commencent à 0 et le dernier vaut length - 1 ; hors limites, la réponse est undefined.
- for...of lit les valeurs une à une ; for classique quand l'indice est nécessaire.
- Objet = paires clé/valeur ; accès par point (etudiant.nom) ou par crochets (etudiant["nom"]).

Exercice

On donne let prix = [500, 1200, 300, 2500, 800]; (des prix en FCFA). 1) Écris le code qui compte combien d'articles coûtent plus de 1000 FCFA et affiche ce nombre. 2) Calcule le résultat attendu à la main. 3) On crée l'objet let produit = { nom: "Riz 5 kg", prix: 2500, stock: 12 }; écris l'instruction qui affiche « Riz 5 kg : 2500 FCFA » à partir de l'objet, puis celle qui baisse le stock de 1 après une vente.

✔ Corrigé
1) let compteur = 0; for (let p of prix) { if (p > 1000) { compteur++; } } console.log(compteur);. Une écriture équivalente utilise filter : prix.filter(p => p > 1000).length. 2) Les prix supérieurs à 1000 sont 1200 et 2500 : le compteur vaut 2. 3) Affichage : console.log(produit.nom + " : " + produit.prix + " FCFA"); — ou, plus lisible, le gabarit `console.log(`${produit.nom} : ${produit.prix} FCFA`);`. Baisse du stock : produit.stock = produit.stock - 1; (ou produit.stock--;), ce qui le fait passer de 12 à 11. L'exercice illustre la différence de rôles : le tableau répond à « combien et lesquels », l'objet décrit « quoi, à quel prix, en quelle quantité ».

Leçon 7 — Projet : page interactive avec validation de formulaire

Assemblons maintenant tout le cours dans un petit projet réaliste : la page d'inscription au club informatique de l'institut. La page contient un formulaire avec trois champs — nom complet, adresse e-mail et niveau (un menu déroulant L1, L2, L3) — un bouton d'envoi, une zone de messages et, sous le formulaire, la liste des membres déjà inscrits qui s'allonge à chaque inscription réussie. Les règles de validation sont simples mais strictes : le nom doit contenir au moins 3 caractères, l'e-mail doit contenir un « @ » et un « . » après lui, et chaque message d'erreur doit apparaître juste sous le champ fautif, en rouge, plutôt que dans une alerte générique que personne ne lit jusqu'au bout.

La démarche suit toujours le même plan, qu'il faut savoir réciter : d'abord sélectionner tous les éléments utiles (formulaire, champs, zones de messages, liste) ; ensuite attacher un gestionnaire à l'événement submit du formulaire, qui commence par preventDefault() ; puis lire les valeurs avec .value en les nettoyant (trim() enlève les espaces en trop au début et à la fin) ; appliquer chaque règle de validation dans une petite fonction séparée — par exemple emailValide(adresse) qui renvoie un booléen — et écrire le message correspondant sous le champ concerné ; enfin, si tout est valide, ajouter le membre à la liste affichée, ranger ses données dans un objet glissé dans un tableau membres, remercier l'utilisateur par un message de succès et vider le formulaire avec reset(). Découper ainsi le travail en petites fonctions testées une à une est ce qui rend le projet abordable.

Deux remarques de professionnel pour terminer. D'abord, chaque règle doit être testée dans les deux sens : un e-mail correct doit passer, un e-mail sans @ doit être refusé — tester seulement le cas qui marche laisse la moitié des bogues en place. Ensuite, il faut connaître la limite honnête de ce que nous faisons : la validation côté client sert au confort de l'utilisateur, qui est corrigé immédiatement et gentiment ; elle ne protège pas le site, car un visiteur déterminé peut toujours contourner le JavaScript de son navigateur. Sur un vrai service, le serveur revérifie donc toutes les données à l'arrivée. Pour notre club, en revanche, cette page suffit largement — et elle prouve que tu sais désormais faire dialoguer HTML, DOM, événements et données.

L'essentiel
Le projet combine tout : sélectionner, écouter submit, preventDefault(), lire et nettoyer les valeurs, valider règle par règle avec des fonctions dédiées, afficher des erreurs précises sous les champs, puis enregistrer le membre dans un tableau d'objets et l'ajouter à la liste. La validation côté client améliore le confort ; seul le serveur peut garantir la sécurité.

Exemple — Formulaire d'inscription au club : structure et validation

```
<form id="formInscription">
<label>Nom complet : <input type="text" id="nom"></label>
<p class="erreur" id="errNom"></p>
<label>E-mail : <input type="text" id="email"></label>
<p class="erreur" id="errEmail"></p>
<label>Niveau :
<select id="niveau">
<option value="">-- Choisir --</option>
<option value="L1">L1</option>
<option value="L2">L2</option>
<option value="L3">L3</option>
</select>
</label>
<p class="erreur" id="errNiveau"></p>
<button type="submit">S'inscrire</button>
</form>
<p id="succes"></p>
<ul id="listeMembres"></ul>

<script>
let membres = [];
function emailValide(adresse) {
let posA = adresse.indexOf("@");
let posPoint = adresse.lastIndexOf(".");
return posA > 0 && posPoint > posA + 1 && posPoint < adresse.length - 1;
}
let form = document.getElementById("formInscription");
form.addEventListener("submit", function (event) {
event.preventDefault();
let nom = document.getElementById("nom").value.trim();
let email = document.getElementById("email").value.trim();
let niveau = document.getElementById("niveau").value;
let errNom = document.getElementById("errNom");
let errEmail = document.getElementById("errEmail");
let errNiveau = document.getElementById("errNiveau");
errNom.textContent = ""; errEmail.textContent = ""; errNiveau.textContent = "";
let valide = true;
if (nom.length < 3) { errNom.textContent = "Nom : au moins 3 caractères."; valide =
false; }
if (!emailValide(email)) { errEmail.textContent = "E-mail invalide (ex. :
nom@exemple.ml)."; valide = false; }
if (niveau === "") { errNiveau.textContent = "Choisis ton niveau."; valide = false; }
if (!valide) { return; }
membres.push({ nom: nom, email: email, niveau: niveau });
let item = document.createElement("li");
item.textContent = nom + " (" + niveau + ") — " + email;
document.getElementById("listeMembres").appendChild(item);
document.getElementById("succes").textContent = "Bienvenue au club, " + nom + " !";
form.reset();
});
</script>
```

À retenir

• Plan type d'une validation : lire, nettoyer (trim), tester chaque règle, afficher l'erreur sous le champ fautif.

• Une fonction par règle (emailValide…) rend le code testable et réutilisable.

• Côté client = confort et rapidité ; la sécurité réelle exigerait une revérification côté serveur.

Exercice

On ajoute au formulaire du projet un champ téléphone <input type="text" id="tel"> avec son paragraphe d'erreur errTel. La règle : le numéro doit contenir exactement 8 chiffres (contexte malien, sans indicatif). 1) Écris la fonction telephoneValide(tel) qui renvoie un booléen : elle doit vérifier la longueur et que chaque caractère est un chiffre (aide : parcourir la chaîne et tester si chaque caractère est entre "0" et "9", ou utiliser une boucle avec isNaN sur chaque caractère converti). 2) Montre les lignes à ajouter dans le gestionnaire submit pour appliquer cette règle et afficher « Numéro invalide : 8 chiffres attendus. » sous le champ. 3) La fonction accepte-t-elle "73 25 58 73" ? Explique et propose un nettoyage avant le test.

✔ Corrigé

1) function telephoneValide(tel) { if (tel.length !== 8) { return false; } for (let c of tel) { if (c < "0" || c > "9") { return false; } } return true; } — la boucle rejette dès le premier caractère non numérique ; si elle se termine, les 8 caractères sont des chiffres. 2) Dans le gestionnaire : récupérer let tel = document.getElementById("tel").value.trim(); vider errTel au départ comme les autres ; puis if (!telephoneValide(tel)) { document.getElementById("errTel").textContent = "Numéro invalide : 8 chiffres attendus."; valide = false; } ; et inclure tel dans l'objet membre quand tout est valide. 3) Non : "73 25 58 73" contient 11 caractères dont des espaces, le test de longueur échoue immédiatement. Or l'utilisateur a raison de grouper les chiffres deux par deux, c'est la présentation habituelle. Nettoyage avant le test : tel = tel.replaceAll(" ", ""); pour supprimer les espaces, puis appliquer telephoneValide sur la version nettoyée. Moralité : valider, c'est aussi décider ce que l'on pardonne à la saisie — et le nettoyage (trim, espaces internes) fait partie de la règle métier, pas d'un détail.

Travaux dirigés (TD)

TD 1 — Mini-calculatrice web

Construis une page contenant deux champs de nombre (ids n1 et n2), quatre boutons (ids btnAdd, btnSous, btnMul, btnDiv) et un paragraphe de résultat (id resultat). Écris le JavaScript qui, au clic sur chaque bouton, lit les deux valeurs, effectue l'opération correspondante et affiche le résultat sous la forme « 12 + 5 = 17 ». La division par zéro doit afficher « Division par zéro impossible » au lieu d'un résultat. Factorise au maximum : une seule fonction calculer(operation) appelée par les quatre boutons est attendue, plutôt que quatre gestionnaires presque identiques.

✔ Corrigé du TD 1
Sélections : const n1 = document.getElementById("n1"), n2, resultat de la même façon. Fonction centrale : function calculer(op) { let a = Number(n1.value); let b = Number(n2.value); let r; if (op === "+") { r = a + b; } else if (op === "-") { r = a - b; } else if (op === "*") { r = a * b; } else { if (b === 0) { resultat.textContent = "Division par zéro impossible"; return; } r = a / b; } resultat.textContent = a + " " + op + " " + b + " = " + r; }. Branchements : btnAdd.addEventListener("click", function () { calculer("+"); }); et de même pour les trois autres. Points de vigilance du corrigé : la conversion Number() est indispensable, sinon le bouton + concaténerait les textes ("12" + "5" = "125") alors que les autres opérations convertiraient toutes seules — incohérence classique ; le test de division par zéro se fait avant le calcul ; factoriser dans calculer(op) évite de recopier quatre fois la lecture des champs, donc quatre occasions d'oublier une conversion.

TD 2 — Liste de tâches interactive

On veut une mini application « mes tâches » : un champ de texte (id nouvelleTache), un bouton « Ajouter » (id btnAjouter), une liste vide <ul id="liste"> et un paragraphe compteur (id compteur). À chaque clic sur Ajouter : si le champ est vide (après trim), ne rien faire et afficher « Écris d'abord une tâche. » dans le compteur ; sinon, créer un élément de liste contenant le texte saisi, l'ajouter à la liste, vider le champ, et mettre à jour le compteur (« 3 tâche(s) en cours »). Bonus : cliquer sur une tâche de la liste la barre (classe CSS "faite", qui met le texte en barré) ou la dé-barre si on reclique.

✔ Corrigé du TD 2
let taches = []; function majCompteur() { compteur.textContent = taches.length + " tâche(s) en cours"; }. Gestionnaire du bouton : let texte = champ.value.trim(); if (texte === "") { compteur.textContent = "Écris d'abord une tâche."; return; } let item = document.createElement("li"); item.textContent = texte; item.addEventListener("click", function () { item.classList.toggle("faite"); }); liste.appendChild(item); taches.push(texte); champ.value = ""; majCompteur();. Le gestionnaire de clic est attaché à chaque item au moment de sa création : chacun barre son propre texte grâce à classList.toggle, qui ajoute la classe si elle absente et la retire si présente. Note que le tableau taches sert ici de modèle des données (il compte les tâches), pendant que le DOM sert d'affichage : garder les deux synchronisés est le principe de toutes les applications web, même les plus grosses.

Travaux pratiques (TP)

TP 1 — Page de présentation personnelle interactive

En t'appuyant sur tout le cours, construis une page de présentation personnelle complète dans un seul fichier HTML (styles dans le <head>, script en fin de <body>). La page doit contenir : un titre avec ton nom, un paragraphe de présentation, un bouton « Mode sombre / Mode clair » qui bascule une classe sombre sur le body (à définir en CSS), un compteur de « visites » qui augmente à chaque clic sur un bouton « J'aime » et s'affiche dans la page, et un mini formulaire de contact (nom, message) validé en JavaScript : nom d'au moins 3 caractères, message d'au moins 10 caractères,

erreurs sous les champs, message de remerciement personnalisé en cas de succès et réinitialisation du formulaire. Tout doit être testé dans le navigateur, console ouverte.

✔ Attendus du TP 1
Attendus : 1) structure propre, ids cohérents entre HTML et JavaScript, aucun message rouge dans la console sur une utilisation normale. 2) Mode sombre : bouton.addEventListener("click", ...) avec document.body.classList.toggle("sombre") ; le CSS définit .sombre { background-color: #222; color: #eee; } et le libellé du bouton peut changer selon l'état. 3) Compteur : une variable let likes = 0, incrémentée au clic, affichée dans un via textContent. 4) Formulaire : gestionnaire submit avec preventDefault(), lecture des .value, trim, deux règles de longueur avec messages sous les champs, puis en cas de succès un texte du type « Merci Awa, ton message est bien noté ! » construit à partir du nom saisi, et form.reset(). 5) Tests à effectuer et à noter : envoyer le formulaire vide (deux erreurs, aucun rechargement), corriger un seul champ (l'erreur correspondante disparaît au prochain envoi), envoyer un formulaire correct (remerciement affiché, champs vidés), cliquer dix fois sur « J'aime » (compteur à 10), basculer le mode sombre deux fois (retour à l'état initial). Un TP réussi est un TP dont chaque fonction a été provoquée volontairement dans le bon et le mauvais cas.

Exercices corrigés supplémentaires

Exercice 1

Écris une fonction comparerAuSecret(proposition) pour un jeu de devinette : le nombre secret est 7 (constante dans la fonction). La fonction renvoie "Trop petit" si la proposition est inférieure, "Trop grand" si elle est supérieure, "Bravo, trouvé !" si elle est égale. Teste-la avec 3, 12 et 7.

✔ Corrigé
function comparerAuSecret(proposition) { const SECRET = 7; if (proposition < SECRET) { return "Trop petit"; } else if (proposition > SECRET) { return "Trop grand"; } else { return "Bravo, trouvé !"; } }. Tests : comparerAuSecret(3) renvoie "Trop petit", comparerAuSecret(12) renvoie "Trop grand", comparerAuSecret(7) tombe dans le else final et renvoie "Bravo, trouvé !". La structure en cascade dispense de tester l'égalité explicitement : si ce n'est ni plus petit ni plus grand, c'est forcément égal — raisonnement par élimination typique des conditions enchaînées.

Exercice 2

On donne let noms = ["Awa", "Moussa", "Ali", "Fanta", "Aminata"]; et une liste vide <ul id="resultat"> dans la page. Écris le code qui n'affiche dans la liste que les prénoms commençant par la lettre « A ». (Aide : le premier caractère d'une chaîne s'obtient par nom[0].) Donne le contenu final de la liste.

✔ Corrigé
let liste = document.getElementById("resultat"); for (let nom of noms) { if (nom[0] === "A") { let item = document.createElement("li"); item.textContent = nom; liste.appendChild(item); } }. La liste finale contient trois éléments : Awa, Ali et Aminata ; Moussa et Fanta sont écartés par le test. On retrouve le schéma universel du filtrage : parcourir une collection, tester chaque élément, ne conserver (ici, ne créer) que ceux qui passent le test — exactement ce que fait la méthode filter, que l'on pourrait écrire : noms.filter(n => n[0] === "A") pour obtenir le tableau filtré avant de l'afficher.

Exercice 3

Reprends la fonction emailValide du projet : elle exige un « @ » après au moins un caractère, un « . » après le @, et au moins un caractère après le point. Sans coder, classe ces adresses en valides ou invalides selon CETTE règle, en justifiant chaque refus : a) "awa@iam.ml" ; b) "awa.iam.ml" ; c) "@iam.ml" ; d) "awa@iam" ; e) "awa@iam." ; f) "a@b.c".

✔ Corrigé
a) Valide : @ en position 3, point après lui, « ml » après le point. b) Invalide : aucun @, indexOf("@") renvoie -1, le test posA > 0 échoue. c) Invalide : le @ est le tout premier caractère (posA = 0), il manque la partie nom avant lui. d) Invalide : aucun point après le @, lastIndexOf(".") renvoie -1 ou un point situé avant le @, le test échoue. e) Invalide : le point est le dernier caractère, il ne reste rien après lui (posPoint < adresse.length - 1 est faux) — une extension de domaine vide n'a pas de sens. f) Valide selon notre règle : un caractère avant le @, un point après, un caractère après le point ; la règle est volontairement simple et accepterait donc cette adresse minimale. Cet exercice montre qu'une validation est un compromis : assez stricte pour attraper les fautes de frappe courantes, assez simple pour être écrite et testée par un humain.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Pour qu'un script externe ne cherche pas des éléments pas encore chargés, on le place…

   A) Dans le <head>, sans précaution

   B) Juste avant </body>, ou avec l'attribut defer

   C) Après la balise </html>

2. Que vaut l'expression "5" + 3 en JavaScript ?

   A) 8

   B) "53"

   C) Une erreur

3. Quelle déclaration crée une variable que l'on ne pourra pas réassigner ?

A) const TAUX = 0.18;

B) let TAUX = 0.18;

C) var TAUX = 0.18;

4. L'opérateur === compare…

A) Uniquement après avoir converti les types

B) La valeur seulement

C) La valeur ET le type, sans conversion

5. Pour sélectionner l'élément d'id "resultat", on écrit…

A) document.querySelector(".resultat")

B) document.getElementById("resultat")

C) document.getElementByClass("resultat")

6. Quelle propriété lit le texte tapé dans un champ <input id="nom"> ?

A) nom.textContent

B) nom.value

C) nom.innerHTML

7. Dans un gestionnaire d'événement submit, event.preventDefault() sert à…

A) Empêcher le rechargement de la page à l'envoi du formulaire

B) Envoyer le formulaire plus vite

C) Vider tous les champs

8. Dans let t = [10, 20, 30];, que vaut t[1] ?

A) 10

B) 30

C) 20

9. Comment ajoute-t-on une valeur à la fin d'un tableau notes ?

A) notes.add(14)

B) notes.push(14)

C) notes[notes.taille] = 14

10. Pour lire la propriété nom de l'objet etudiant, on écrit…

A) etudiant->nom

B) etudiant[nom]

C) etudiant.nom

Corrigés du QCM

1. Réponse B — Le navigateur exécute le code au fil de la lecture : en fin de body (ou avec defer), tout le HTML existe déjà.

2. Réponse B — Dès qu'un côté du + est une chaîne, l'opération devient une concaténation : le 3 est collé au texte.

3. Réponse A — const fixe la référence : toute nouvelle affectation déclenche une erreur. let et var autorisent la réaffectation.

4. Réponse C — === est l'égalité stricte : "5" === 5 est faux car les types diffèrent, alors que "5" == 5 est vrai.

5. Réponse B — getElementById vise un id unique. querySelector ferait aussi l'affaire avec le sélecteur "#resultat".

6. Réponse B — La valeur courante d'un champ se lit (et s'écrit) avec .value ; elle est toujours de type chaîne.

7. Réponse A — Sans lui, le navigateur recharge la page et la validation JavaScript n'a pas le temps d'agir visiblement.

8. Réponse C — Les indices commencent à 0 : t[0] = 10, t[1] = 20, t[2] = 30.

9. Réponse B — push ajoute un élément en fin de tableau et met à jour length automatiquement.

10. Réponse C — La notation pointée etudiant.nom est la forme standard ; etudiant["nom"] est l'équivalent par crochets.

Résumé — les notions importantes

• JavaScript s'insère par <script> ou par fichier .js, s'exécute dans l'ordre de la page, et se débogue dans la console (console.log, erreurs en rouge).

• let pour les valeurs qui changent, const pour les constantes ; types de base : nombre, chaîne, booléen — et les saisies de formulaire arrivent toujours en chaînes, à convertir avec Number().

• Le + concatène dès qu'une chaîne est présente, et === compare valeur et type sans conversion : c'est l'égalité à utiliser par défaut.

• if / else et switch pour décider, for et while pour répéter, fonctions avec paramètres et return pour découper le code en morceaux réutilisables.

• Le DOM représente la page en arbre d'objets : getElementById et querySelector sélectionnent, textContent, classList, style et appendChild modifient.

• Les événements (click, input, submit…) déclenchent des fonctions attachées par addEventListener ; preventDefault() bloque le comportement par défaut d'un formulaire.

• Tableaux (indices dès 0, length, push, parcours) et objets (paires clé/valeur) structurent les données ; combinés, ils permettent la validation complète d'un formulaire côté client.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Apprendre JavaScript en 1h : Cours complet débutant 2026

Chaîne : Comment Coder · Langue : Français · Niveau : Débutant

Un cours d'ensemble d'une heure pour démarrer : où écrire le code, variables, conditions, boucles et premières fonctions, avec des exemples pratiques. Parfait en ouverture du cours, avant d'entrer dans le détail de chaque leçon.

https://www.youtube.com/watch?v=ldxt9FNUc50

Leçon 2 — Cours JavaScript pour les débutants – Bases essentielles expliquées

Chaîne : ETC SCHOOL · Langue : Français · Niveau : Débutant

Les fondamentaux expliqués pas à pas : console.log, déclarations let et const, types de données, conditions if/else, fonctions et boucles for. Le tout est illustré par un fil rouge concret, une mini-boutique en ligne, proche de nos exemples de calculs de prix.

https://www.youtube.com/watch?v=7V3EN41zSdY

Leçon 3 — Introduction Complète à JavaScript : Variables, Fonctions, Conditions et Plus !

Chaîne : Hamza Ma · Langue : Français · Niveau : Débutant

Un panorama structuré des bases : variables et opérateurs, structures de contrôle (conditions et boucles), puis un premier contact avec la manipulation du DOM. Un bon complément vidéo pour consolider les leçons 2 et 3 avant d'aborder le DOM.

https://www.youtube.com/watch?v=p11-Z70AGz4

Leçon 4 — Maîtrise JavaScript et le DOM en 30 Minutes - Tutoriel Complet Débutants (2026)

Chaîne : Prakticode · Langue : Français · Niveau : Débutant

La manipulation du DOM en pratique : créer et styliser des éléments dynamiquement, construire une navigation interactive et un compteur cliquable pour la gestion des événements. À regarder en travaillant les leçons 4 et 5, exemples à recopier en parallèle.

https://www.youtube.com/watch?v=L0P1sH3FGi4

Leçon 6 — JavaScript Moderne -- Cours complet pour débutants dans seulement 3 heures

Chaîne : Malakisi · Langue : Français · Niveau : Débutant à intermédiaire

Un cours long et structuré qui couvre variables, fonctions, tableaux et objets, puis la manipulation du DOM et l'organisation du code en modules. Idéal comme révision complète en fin de cours et pour approfondir la leçon 6 sur les tableaux et les objets.

https://www.youtube.com/watch?v=tcf1hvbByWU

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Documentation MDN Web Docs (developer.mozilla.org/fr) — les références de JavaScript et du DOM, gratuites et en français.

• Tutoriel « Le guide JavaScript moderne » (fr.javascript.info) — un parcours complet et progressif, consultable en ligne.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

• Ressources des bibliothèques universitaires francophones en accès libre sur le développement web — demande les références à jour à la bibliothèque de l'IAM.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !